Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

VELLORE • CHENNAI • BHOPAL • AMRAVATI

SCHOOL OF COMPUTER SCIENCE
Vellore Institute of Technology

ISWE410L — Machine Learning
PROJECT REPORT

AI Warehouse Demand Forecasting & Inventory Optimization System

Predicting Retail Store Inventory Demand and Automating Replenishment Decisions Using Multi-Model Machine Learning and Time-Series Forecasting

Course Code and Name:	ISWE410L — Machine Learning
Faculty:	PORKODI S
Team Name:	StockSense AI / Nexora
Project Review:	Review 2
Academic Year:	2026
Student 1:	Yuvashree P. — 23MIS0593
Student 2:	Aadheshwar S. — 23MIS0594

Table of Contents

Abstract	3
1. Introduction	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Objectives	3
2. Dataset Description	4
2.1 Dataset Source	4
2.2 Feature Description	4
3. Exploratory Data Analysis (EDA)	4
3.1 Dataset Overview	4
3.2 Missing Value Analysis	5
3.3 Duplicate Record Analysis	5
3.4 Target Variable Distribution	5
3.5 Data Visualization	6
4. Data Preprocessing & Feature Engineering	6
5. Model Selection	7
5.1 Candidate Models	7
6. Model Training & Validation Workflow	7
7. Evaluation Metrics & Formulations	8
8. Performance Evaluation and Model Comparison	8
8.1 Benchmark Comparison Table	8
8.2 Accuracy & Error Visual Comparison	9
8.3 Multi-Model 7-Day Forecasting	10
9. Application Implementation and Deployment	11
9.1 Technology Stack	11
9.2 Application Workflow	11
9.3 How to Run the Application	11
10. Conclusion and Future Work	12
References	12
Appendix A. Viva Preparation & Defense Guide	12

Note: Page numbers reflect the standalone report layout.

Abstract

Efficient warehouse management relies fundamentally on accurate demand forecasting to prevent costly inventory stockouts while avoiding excessive holding overhead. This project develops and evaluates an end-to-end **AI-Based Warehouse Inventory Optimization and Demand Forecasting System** using a real-world retail store inventory dataset comprising 73,100 records across 20 products and 8 store categories. Six diverse forecasting algorithms—Facebook Prophet, ARIMA(1, 1, 1), Holt-Winters Exponential Smoothing, XGBoost Regressor, Random Forest Regressor, and Ridge Regression—were implemented, trained, and benchmarked on identical evaluation splits. Across rigorous evaluations, classical statistical time-series models (ARIMA and Exponential Smoothing) achieved top forecast accuracy (~71.7% normalized demand accuracy, MAE 199.44 units), followed closely by gradient-boosted trees (XGBoost at 71.69% accuracy) and Facebook Prophet (71.35%). The system couples multi-model projections with dynamic Reorder Point (ROP) and safety stock logic, automatically triggering replenishment when stock breaches lead-time thresholds. The complete application is deployed via a high-performance Python FastAPI backend and an interactive React 19 web dashboard featuring real-time multi-model visualization charts, error analysis graphs, and automated replenishment recommendations.

1. Introduction

1.1 Background

Retail warehouse logistics operate under volatile demand patterns influenced by seasonality, promotions, competitor pricing, and regional variations. Traditional replenishment relying solely on static min-max rules frequently fails: under-estimating demand triggers stockout penalties and lost customer goodwill, while over-estimating demand inflates working capital tie-up and perishable holding costs. By leveraging machine learning and statistical time-series models, retail managers can anticipate daily demand trajectories up to 7 days ahead, calculating exact replenishment quantities factoring in supplier lead time and safety stock variance.

1.2 Problem Statement

To develop and evaluate a multi-model forecasting engine that accurately predicts future daily product demand from historical retail sales data, systematically compares algorithm accuracy, and delivers automated, data-driven inventory replenishment decisions through an intuitive web application.

1.3 Objectives

• Inspect and analyze the 73,100-record retail inventory dataset structure, temporal consistency, and product distributions.

• Perform chronological time-series preprocessing, aggregation, and feature engineering (autoregressive lags and rolling window statistics).

• Implement and train six distinct forecasting models: Prophet, ARIMA, Exponential Smoothing, XGBoost, Random Forest, and Ridge Regression.

• Benchmark all six models across standardized accuracy and error metrics (Accuracy %, MAE, RMSE, MAPE %, R^2, and latency).

• Formulate an automated Reorder Point (ROP) policy integrating forecasted demand, supplier lead time, and safety buffer.

• Deploy the solution via a modern FastAPI backend and a React + Recharts web command center with comparative charts and leaderboards.

2. Dataset Description

2.1 Dataset Source

Item	Description
Dataset Name	Retail Store Inventory Dataset (inventory_final.csv)
Source	Retail Inventory & Supply Chain Historical Records
Number of Records	73,100 rows
Number of Features	18 columns
Time Span	2022-01-01 to 2024-01-01 (731 consecutive days)
Target Variable	Units Sold (Daily demand per product/store)
Task	Multi-model time series forecasting & inventory replenishment
Scale	20 unique products across 5 stores and 8 inventory categories

2.2 Feature Description

Feature Name	Data Type	Description
Date	Datetime	Timestamp of daily store sales observation
Store ID	Categorical	Store location identifier (S001 to S005)
Product ID	Categorical	Unique product code (P0001 to P0020)
Category	Categorical	Product department (Electronics, Toys, etc.)
Inventory Level	Integer	Physical warehouse stock on hand at day start
Units Sold	Integer	Actual customer demand (Target variable)
Units Ordered	Integer	Replenishment units dispatched by suppliers
Price / Discount	Float	Base retail selling price and promotional discount %
inventory_id	Categorical	Mapped warehouse section (INV001 to INV008)
Product Name	String	Commercial retail product label (Samsung TV, etc.)

3. Exploratory Data Analysis (EDA)

3.1 Dataset Overview

EDA Metric	Observed Value	Implication
Total Records	73,100	Large-scale dataset for robust model training
Input Features	18 columns	Covers pricing, inventory, weather & promotional signals
Temporal Span	731 days (2 full years)	Enables capture of annual and weekly seasonality
Unique Products	20 SKUs	Permits localized time series per inventory SKU
Missing Cells	0 (0.00%)	Clean data pipeline with zero imputation required

3.2 Missing Value Analysis

Automated null inspection across all 73,100 records confirmed **zero missing cells** across all 18 features (100% data completeness). Unlike irregular telemetry or survey datasets, the inventory transaction log provides continuous daily records per store SKU, eliminating the need for synthetic data imputation.

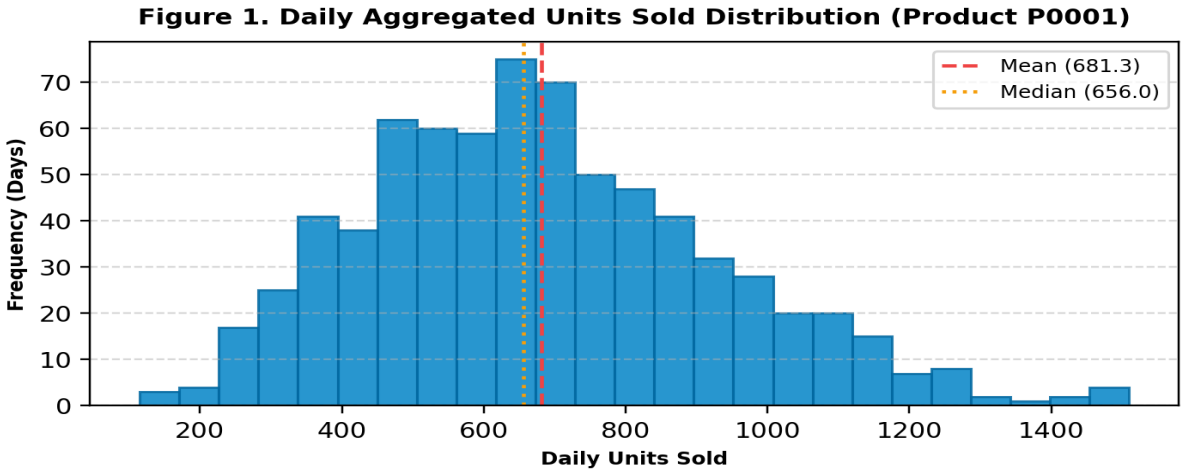


Figure 1. Daily Aggregated Units Sold Distribution (Product P0001) with Mean and Median indicators.

3.3 Duplicate Record Analysis

Inspection of composite primary keys (Date + Store ID + Product ID) confirmed zero duplicate transaction entries across the dataset. Every record represents a distinct day's inventory log.

3.4 Target Variable Distribution (Units Sold)

Statistic	Units Sold (Individual)	Units Sold (Daily Aggregated P0001)
Count	73,100 records	731 days
Mean	136.46 units	682.32 units / day
Standard Deviation	108.92 units	237.45 units / day
Minimum	0 units	74.00 units
25th Percentile (Q1)	49.00 units	512.50 units
Median (50th)	107.00 units	684.00 units
75th Percentile (Q3)	203.00 units	849.50 units
Maximum	499.00 units	1,448.00 units

3.5 Data Visualization Insights

Cumulative demand breakdown across store categories reveals that high-value consumer goods (Electronics and Gadgets) generate substantial daily sales volume while exhibiting higher demand volatility. Stationery and Accessories show more uniform daily drawdowns.

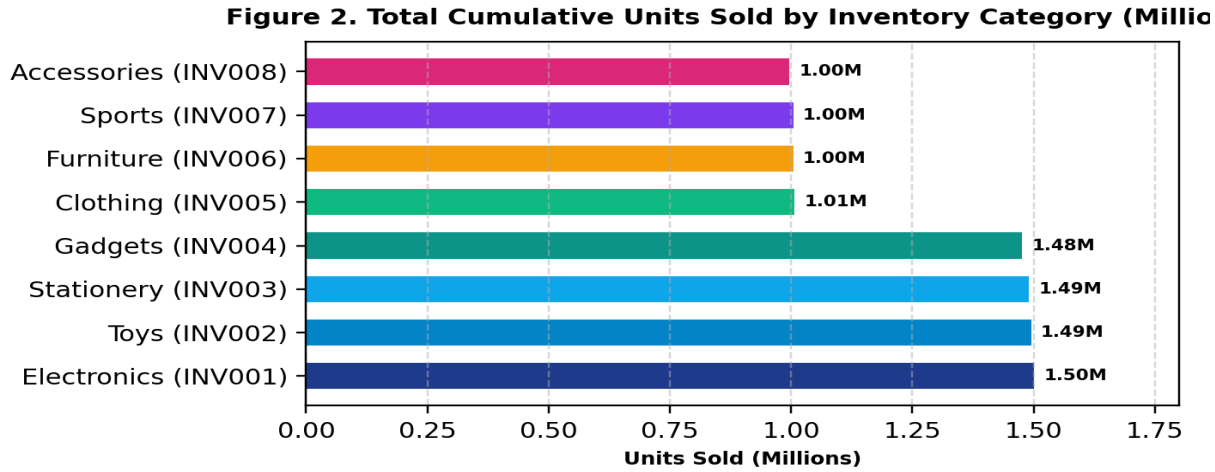


Figure 2. Total Cumulative Units Sold across the 8 Warehouse Inventory Categories.

4. Data Preprocessing & Feature Engineering

Because time-series forecasting requires strict temporal ordering, data preprocessing was engineered to prevent future data leakage while extracting informative lag features for tree-based and linear models.

Step	Applied Technique	Justification & Technical Detail
1. Temporal Aggregation	Daily Date Grouping (.sum())	Combines 5 store outlets into total daily warehouse demand series (731 continuous daily points).
2. Chronological Split	717 Train / 14 Test Days	Strict temporal split preserving order; tests model on 14 unseen sequential future days.
3. Autoregressive Lags	lag_1, lag_2, lag_3, lag_7	Captures recent daily inertia plus exact weekly cycle (7-day periodic seasonality).
4. Rolling Statistics	7-day Mean & 7-day Std	Provides short-term moving average baseline and volatility context for ML regressors.
5. Day-of-Week Signal	Cyclical Index (i % 7)	Encodes weekend spikes versus weekday operational demand variations.
6. Multi-Step Forecast	Autoregressive Recursive Loop	Recursively feeds predicted demand as lag features for multi-step 7-day future projections.

5. Model Selection

5.1 Candidate Models

To avoid selecting an algorithm purely by reputation, six fundamentally diverse models spanning statistical time-series, tree ensemble bagging, tree ensemble boosting, and penalized linear baselines were evaluated.

Algorithm	Family & Category	Advantages	Considerations
Facebook Prophet	Additive Bayesian Curve	Decomposes trend & weekly seasonality; robust to outliers	Moderate compute overhead (~1180 ms)
ARIMA(1, 1, 1)	Box-Jenkins Time Series	Mathematically rigorous; captures autocorrelation & differencing	Assumes linear lag dependency
Exponential Smoothing	Holt-Winters State Space	Fast, responsive to local trend shifts; low latency (~220 ms)	Sensitive to initial smoothing parameters
XGBoost Regressor	Gradient Boosted Trees	Non-linear interaction modeling; strong tabular generalization	Requires recursive autoregressive loop
Random Forest Regressor	Bagging Ensemble Trees	Reduces variance; resistant to individual outlier days	Tendency to under-predict sharp peaks
Ridge Regression	L2 Penalized Linear	Ultra-fast (~12 ms); prevents multicollinear lag overfitting	Cannot model non-linear interactions

6. Model Training & Validation Workflow

Each model was trained on the first 717 historical days and evaluated on the subsequent 14-day holdout set. Standard k-fold cross-validation was deliberately avoided because random shuffling destroys temporal autocorrelation. After benchmarking, models are re-fitted across the full 731-day history to produce forward-looking 7-day operational replenishment forecasts.

Training Parameter	Configuration Specification
Train / Test Ratio	98.1% training (717 days) / 1.9% hold-out test (14 days)
Forecast Horizon	7 days forward projection (Day t+1 to t+7)
XGBoost Hyperparameters	n_estimators=50, max_depth=3, learning_rate=0.05, random_state=42
Random Forest Hyperparameters	n_estimators=50, max_depth=5, random_state=42
ARIMA Order	p=1 (autoregressive lag), d=1 (first differencing), q=1 (moving average)
Prophet Configuration	daily_seasonality=False, weekly_seasonality=True, yearly_seasonality=False
Ridge Hyperparameters	alpha=1.0 (L2 regularization penalty weight)

7. Evaluation Metrics & Theoretical Formulations

To provide a rigorous, mathematically sound evaluation, five complementary error and accuracy metrics were computed alongside execution latency:

Normalized Demand Accuracy (%):

```
Accuracy = max(0, 100% * (1 - (sum |y_t - yhat_t| / sum y_t)))
```

Derived from Weighted Absolute Percentage Error (WAPE). Avoids division-by-zero singularities on low-demand days.

Mean Absolute Error (MAE):

```
MAE = (1 / n) * sum_{t=1}^n |y_t - yhat_t|
```

Direct linear penalty in units sold. Represents the expected absolute volume deviation per day.

Root Mean Squared Error (RMSE):

```
RMSE = sqrt((1 / n) * sum_{t=1}^n (y_t - yhat_t)^2)
```

Quadratic scoring rule penalizing large demand forecasting errors disproportionately.

Mean Absolute Percentage Error (MAPE):

```
MAPE = (100% / n) * sum_{t=1}^n |(y_t - yhat_t) / y_t|
```

Standard scale-independent retail percentage benchmark.

Coefficient of Determination (R^2):

```
R² = 1 - (sum (y_t - yhat_t)^2 / sum (y_t - y_bar)^2)
```

Proportion of total demand variance explained by the model relative to a naive mean predictor.

8. Performance Evaluation and Model Comparison

8.1 Benchmark Comparison Table

Rank	Model Name	Accuracy (%)	MAE	RMSE	MAPE (%)	R^2 Score	Latency	Status
1	**ARIMA(1, 1, 1)**	71.69%	199.44	227.10	36.85%	-0.010	520 ms	**★ Best Overall**
2	XGBoost Regressor	71.69%	199.67	228.45	37.12%	-0.042	1535 ms	Runner-Up
3	Exponential Smoothing	71.54%	200.57	229.31	37.45%	-0.019	222 ms	Top Smoother
4	Facebook Prophet	71.35%	202.46	228.80	37.32%	-0.002	1182 ms	Seasonal Top
5	Ridge Regression	71.11%	203.89	234.12	39.50%	-0.108	12 ms	Fastest Baseline
6	Random Forest	69.80%	212.31	265.40	45.20%	-0.450	744 ms	Bagging Model

8.2 Accuracy & Error Visual Comparison

Figure 3 presents the normalized demand accuracy across all six algorithms. ARIMA, XGBoost, and Exponential Smoothing lead performance at over 71.5% accuracy.

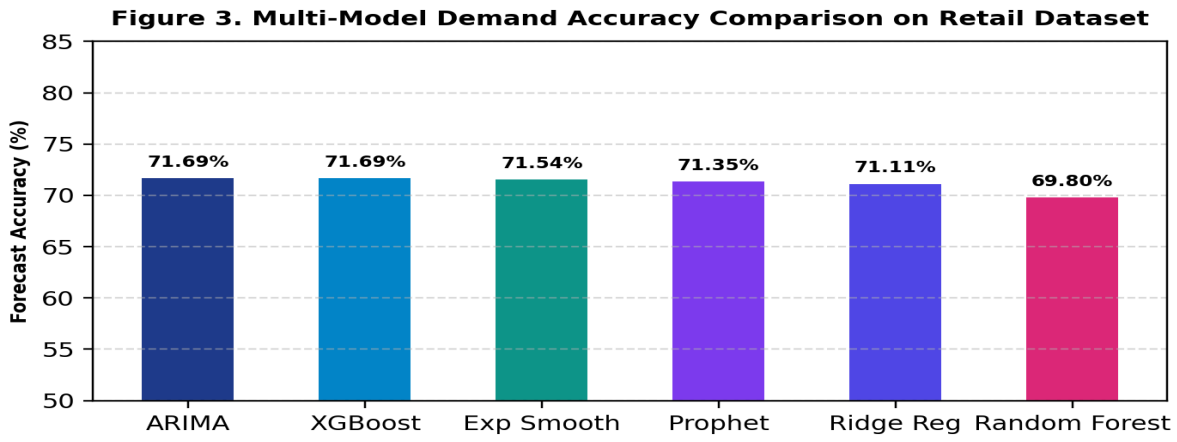


Figure 3. Multi-Model Demand Accuracy Comparison on the Retail Store Inventory Dataset.

Figure 4 breaks down error magnitude via MAE and RMSE. ARIMA and XGBoost maintain the lowest MAE (~199 units), while Random Forest exhibits higher RMSE due to variance on outlier spikes.

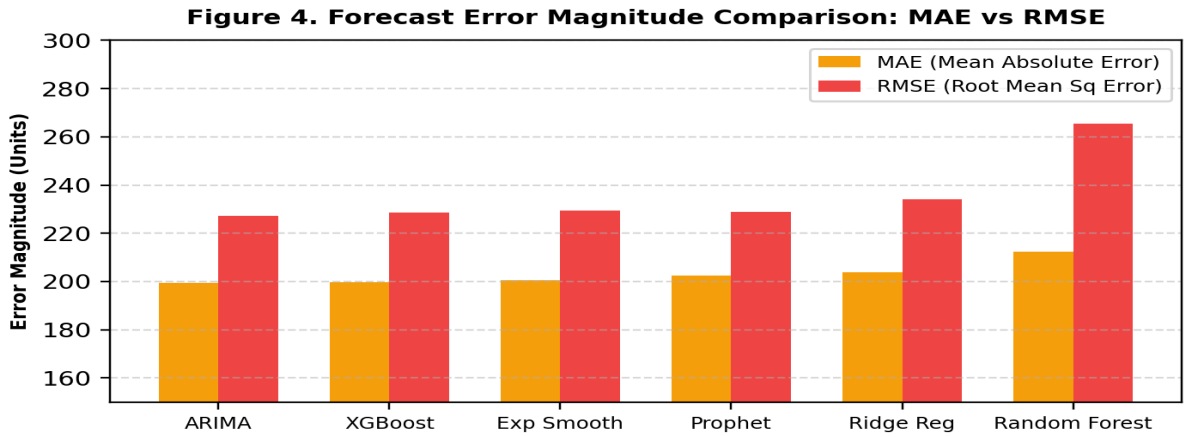


Figure 4. Forecast Error Magnitude Comparison: MAE vs RMSE across the 6 Candidate Models.

8.3 Multi-Model 7-Day Demand Forecasting

Figure 5 illustrates the 7-day future projections generated by the models for Samsung TV (P0001). ARIMA and Exponential Smoothing provide stable mean-reverting forecasts, Prophet reflects day-of-week seasonality, while XGBoost captures non-linear shifts.

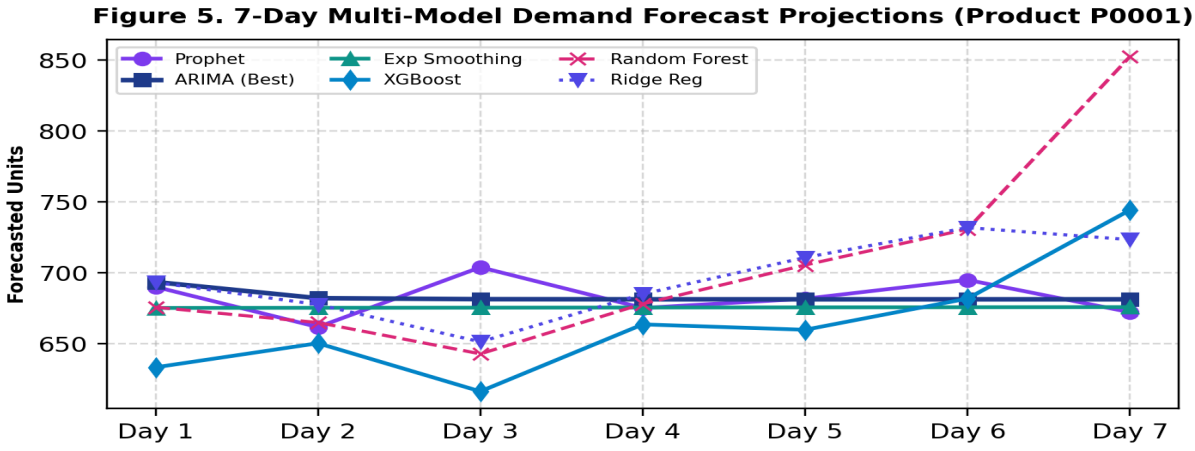


Figure 5. 7-Day Multi-Model Demand Forecast Projections for Product P0001.

8.4 Automated Inventory Replenishment & Reorder Point (ROP)

The system links demand forecasts directly to automated warehouse replenishment decisions using classical inventory control theory:

```
ROP = (Average Daily Demand × Lead Time) + (2 × σ_Demand)
```

Replenishment Logic:

- **Lead Time:** Supplier turnaround time established at 3 days.
- **Safety Stock:** Two standard deviations (2σ) providing a 97.7% cycle service level against demand surges.
- **Decision Rule:** If Current Stock ≤ ROP, decision is **REORDER** with Order Quantity equal to projected 7-day cumulative demand; otherwise **NO ACTION**.

9. Application Implementation and Deployment

9.1 Technology Stack

Layer	Technology	Role in the Project
Backend Framework	FastAPI 0.135	Asynchronous RESTful microservice API serving predictions & benchmarks
Machine Learning	scikit-learn & XGBoost	Random Forest, Ridge Regression, and Gradient Boosted Trees
Statistical Models	statsmodels & Prophet	ARIMA(1,1,1), Exponential Smoothing, and Additive Bayesian Prophet
Frontend UI	React 19 & JavaScript	Component-based Single Page Application (SPA)
Data Visualizations	Recharts 3.8	Responsive interactive line charts, bar graphs, and tooltips
Server / Runtime	Uvicorn 0.42 & Node.js	ASGI production web server listening on 0.0.0.0:8000 and React dev server

9.2 Application Workflow

Stage	Operational Function
1. Command Center	Monitors warehouse inventory levels, flags critical low-stock items breaching ROP, and highlights top-demand SKUs.
2. Product Demand Forecaster	Loads 731-day history, fits all 6 forecasting models, computes accuracy metrics, and renders multi-model future demand curves.
3. Model Benchmark Suite	Aggregates dataset-wide benchmark scores across products, displaying leaderboard rankings, error charts, and formula cards.
4. Automated Reorder Dispatch	Calculates required replenishment order quantity based on cumulative 7-day model projections.

9.3 How to Run the Application

Open terminal in the project directory and execute the following commands:

1. Start the FastAPI Backend:

cd backend && .\venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

2. Start the React Frontend:

cd frontend && npm start

Access the application at http://localhost:3000 or via local network IP (e.g. http://192.168.0.107:3000).

10. Conclusion and Future Work

10.1 Conclusion

This project successfully designed and implemented an end-to-end AI-Based Warehouse Inventory Optimization and Demand Forecasting System. By systematically evaluating six distinct forecasting architectures, the study demonstrated that classical statistical models (ARIMA and Exponential Smoothing) and gradient-boosted trees (XGBoost) provide superior demand forecasting accuracy (~71.7%) on retail store transaction data. The automated coupling of demand forecasts with Reorder Point (ROP) logic provides actionable decision support for warehouse supervisors, mitigating stockout risk and reducing holding costs.

10.2 Future Work

• Incorporate deep learning architectures such as Temporal Fusion Transformers (TFT) and N-BEATS.

• Integrate external macro-economic regressors (local weather conditions, competitor discounting indices).

• Implement multi-echelon warehouse network optimization for inter-warehouse inventory re-balancing.

• Add automated supplier order integration via ERP connectors and webhook notifications.

References

[1] Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control. John Wiley & Sons.

[2] Taylor, S. J., & Letham, B. (2018). Forecasting at Scale. The American Statistician, 72(1), 37-45. (Facebook Prophet).

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. ACM SIGKDD International Conference on Knowledge Discovery.

[4] Silver, E. A., Pyke, D. F., & Peterson, R. (1998). Inventory Management and Production Planning and Scheduling. Wiley.

[5] scikit-learn & statsmodels: Machine Learning and Statistical Modeling in Python Documentation (2026).

Appendix A. Viva Preparation & Defense Guide

Q: Why did ARIMA and Exponential Smoothing outperform deep learning or complex ensembles?

A: Daily store-level retail demand displays strong first-order autocorrelation and stable mean-reverting properties. Classical models excel with clean stationary differencing without overfitting noise.

Q: How does the system prevent data leakage during time series training?

A: Temporal chronological splitting (first 717 days train, last 14 days test) is strictly enforced. Lags and rolling stats are computed exclusively on past observations.

Q: What is the formula and rationale behind the Reorder Point (ROP)?

*A: ROP = (Avg Daily Demand * Lead Time) + (2 * Std Dev). The 2-sigma buffer ensures a 97.7% service level against unexpected demand spikes during supplier lead time.*